aware.py × Terminal

```
1  import os
2  import openai
3  import secret
4  openai.api_key='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'
5
6  message=[
7          {"role": "system", "content": "You are a coding assistant with expert
8          {"role": "user", "content": "I have this JavaScript code: `function a
9      ]
10
11 message=[
12 {"role": "system", "content": "You are a coding assistant with expertise in P
13 {"role": "user", "content": "Explain the following Python code: `def square(x
14     ]
15
16 message = {"role": "system", "content": "You are a coding assistant with expe
17 {"role": "user", "content": "I have this Python code that throws an IndexErro
18
19 message = [
20   {"role": "system", "content": "You are a coding assistant with expertise in
21         {"role": "user", "content": "What are some best practices for naming
22     ]
23
24 # FREEZE CODE BEGIN
25 response=openai.ChatCompletion.create(
26   model="gpt-3.5-turbo",
27   messages= message
28 )
29
30 print(response['choices'][0]['message']['content'].strip())
31
32 # FREEZE CODE END
```

54% (17:48)

Guide ×

Collapse

Best Practices

Using the ChatGPT API for best practices and code optimization can help developers improve their coding skills and enhance the quality of their code. By employing effective techniques for requesting guidance and working with the provided suggestions, you can elevate your programming expertise and create more efficient, maintainable, and robust code.

Consider the following API call to obtain guidance on Python naming conventions:

```
[
  {"role": "system", "content": "You are a coding assistant with expertise in
        {"role": "user", "content": "What are some best practices for naming
    ]
```

TRY IT!

```
Terminated by timeout
```

In this example, the user query explicitly requests guidance on Python naming conventions. The system message sets the context of the AI as a coding assistant with expertise in Python.

Now consider the following API call to optimize a JavaScript code snippet:

```
[
        {"role": "system", "content": "You are a coding assistant with expert
        {"role": "user", "content": "I have this JavaScript code: `function a
    ]
```

TRY IT!

```
Terminated by timeout
```

In this example, the user query provides a JavaScript code snippet and requests suggestions for improvements or optimizations. The system message sets the context of the AI as a coding assistant with expertise in JavaScript.

Techniques for Requesting Best Practices and Optimization:

1. **Be specific**: Clearly describe the area of code or programming concept for which you need guidance on best practices.
2. **Provide context**: Include any relevant background information, constraints, or requirements that may affect the implementation of best practices or optimization.
3. **Use system messages:** Set the context of the AI as a coding assistant with expertise in the programming language or domain you are working with.

Best practices and optimization

True or False:
When requesting best practices and optimization, you should be vague in your queries to allow the AI to explore a wider range of solutions.

- ○ True
- ◉ False

Being specific in your queries can lead to more precise and helpful responses from the AI. Vague queries may lead to more generalized or less relevant responses.

Check It!